

ANOMALÍAS vs GUARDIANES DEL ESPACIO TIEMPO

Planificación de Proyecto · TFG

Distribución de tareas por miembro y sprints

1. RESUMEN EJECUTIVO

El proyecto consiste en desarrollar un juego mobile RPG gacha (Android/iOS) con Kivy (Python) en un plazo de 10 semanas, organizado en 5 Sprints de 14 días con entrega y revisión del profesor al final de cada uno. El equipo está formado por 4 miembros con roles especializados.

Parámetro	Valor
Duración total	~10 semanas (5 Sprints × 14 días)
Revisiones con profesor	Cada 14 días, al cerrar cada Sprint
Plataforma objetivo	Android / iOS con Kivy (Python)
Base de datos	SQLite (local) + Firebase (stats de enemigos)
Personajes totales	12 (6 por facción, 3 rarezas: B / A / S)
Sistema de combate	1vs1 auto-battle con animaciones estilo Darkest Dungeon
Sistema de progresión	Gacha + runas + mapa de nodos estilo Candy Crush

2. ROLES Y RESPONSABILIDADES

A continuación se detalla qué hace cada miembro del equipo, con las funciones ajustadas y equilibradas según el GDD.

M1 — Base de Datos & Backend de Datos

Responsable de toda la capa de persistencia local (SQLite) y del modelo de datos que alimenta el resto del juego.

Función	Descripción
Diseño del esquema SQLite	Tablas: personajes, armas, runas, inventario_jugador, progreso_mapa, contadores_pity, recursos_jugador
CRUD de personajes	Alta, consulta y actualización de personajes del jugador (facción, rareza, clase, stats base)
CRUD de armas y runas	Gestión de armas básicas/S y runas genéricas/únicas, incluyendo relación personaje↔arma S
Gestión de inventario	Registro de qué personajes/armas/runas tiene el jugador, cuáles están equipadas
Persistencia del pity	Guardar y leer el contador de pity por banner (personajes / armas) entre sesiones
Persistencia de recursos	Tickets de personaje, tickets de arma, moneda premium y pociones del jugador
Progreso del mapa	Estado de cada nodo: BLOQUEADO / DISPONIBLE / COMPLETADO + estrellas obtenidas
Seed de datos iniciales	Poblar la BD con los 12 personajes, 7 armas y todas las runas al primer arranque
API interna de datos	Funciones Python reutilizables por M2 y M3: get_personaje(), set_equipo(), add_recurso()...
Integración Firebase (lectura)	Leer stats de enemigos por nodo desde Firebase Realtime DB (sin escritura desde cliente)

M2 — Lógica de Juego: Combate, Stats & Gacha

Responsable de toda la lógica de negocio: simulación de combates, fórmulas de daño, sistema gacha y economía de recursos.

Función	Descripción
Cálculo de stats finales	Sumar stats base + bonuses de arma + bonuses de 2 runas → stats efectivos del personaje activo
Fórmula daño Guerrero/Mago	$DAÑO = STAT_PRINCIPAL \times (100 / (100 + DEF_enemigo))$
Fórmula daño Asesino	$DAÑO = ATK \times factor_destreza$, donde factor $\in [0.5, 2.0]$ ponderado por DESTREZA
Motor de combate (simulación)	Resolver turno a turno hasta 0 PV o límite de turnos; generar log {turno, atacante, daño, PV_restantes}
Resultado de combate	Devolver victoria/derrota + log al front (M3) para animar

Función	Descripción
Sistema gacha — tiradas	Lógica de 1 tirada: generar rareza (B/A/S) según rates configuradas, elegir ítem al azar dentro de la rareza
Sistema gacha — pity	Leer contador de pity de SQLite (M1), incrementar, forzar S al llegar a N (5 en demo, 90 en producción)
Sistema gacha — banners	Separar pool personajes vs pool armas; aplicar pity independiente por banner
Gestión de pociones	Consumir poción al repetir nodo; regeneración automática por tiempo; compra con moneda premium
Validación de equipamiento	Verificar que el arma S asignada solo la puede equipar su personaje S correspondiente
Gestión de recompensas	Al completar nodo: calcular tickets + moneda según tabla de recompensas y llamar a M1 para guardar
Configuración de rates	Fichero de configuración editable sin tocar código (fácil ajuste para la demo del tribunal)

M3 — Interfaz (Kivy) & Arte

Responsable de todas las pantallas del juego y del arte visual. Su trabajo se mantiene tal como estaba planeado.

Función	Descripción
Pantalla de inicio / Home	Mostrar personaje activo con su arma y runas equipadas. Navegación al resto de pantallas
Pantalla de inventario	Lista de personajes, armas y runas. Cambio de personaje activo. Equipo/desequipo de ítems
Pantalla de gacha	Dos banners (personajes/armas), botón de tirada x1, animación de resultado, historial
Pantalla de mapa	Nodos estilo Candy Crush, estados visual (bloqueado/disponible/completado), acceso al combate
Pantalla de combate	Animación 2D lateral estilo Darkest Dungeon usando el log generado por M2. Pantalla de resultado
Pantalla de selección de facción	Primera pantalla relevante tras tutorial: elegir Anomalías o Guardianes
Arte de personajes y enemigos	Sprites 2D para los 12 personajes + enemigos del mapa
Arte de armas y runas	Iconos de las 7 armas y todas las runas
Fondos y UI	Fondos por pantalla, botones, marcos, efectos visuales de combate
Animaciones de combate	Secuencia de ataque/daño/derrota reproduciendo el log de M2

M4 — Integración, QA & Firebase

M4 tiene un rol de integrador y asegurador de calidad. Une el trabajo de M1, M2 y M3, gestiona Firebase y garantiza que el juego funcione como un todo. También apoya en las áreas con más carga en cada sprint.

Función	Descripción
Configuración de Firebase	Crear proyecto Firebase, configurar Realtime Database, definir estructura JSON de enemigos por nodo
Gestión de datos de enemigos	Poblar y mantener los datos de PV, DEF y ATK de cada enemigo en Firebase (ajustable sin recompilar)
Integración M1↔M2	Conectar las llamadas de M2 (gacha, combate, recompensas) con la capa de datos de M1
Integración M2↔M3	Asegurar que las pantallas de M3 consumen correctamente los resultados de M2 (log de combate, tiradas)
Testing funcional	Probar cada sistema al cerrarse: gacha, combate, pity, rutas del mapa, equipo/desequipo
Detección y reporte de bugs	Registrar bugs encontrados, asignarlos al miembro responsable y verificar su corrección
Balanceo de stats	Proponer y probar valores numéricos de personajes, armas, runas y enemigos junto con M2
Definición de pendientes del GDD	Liderar las reuniones para resolver las decisiones abiertas (rates gacha, stats base, recompensas por nodo)
Documentación técnica	Mantener actualizado el README del repositorio y los comentarios de las funciones críticas
Demo para el tribunal	Preparar el flujo de demostración (pity a 5, personaje S, combate avanzado) y verificar que funciona

3. PLANIFICACIÓN POR SPRINTS

El proyecto se divide en 5 Sprints de 14 días. Al final de cada Sprint se realiza una revisión con el profesor para obtener el visto bueno antes de continuar.

SPRINT 1 Días 1–14

□ **Objetivo:** Fundamentos: BD, arquitectura base y pantalla de selección de facción

Decisiones previas al Sprint 1 (semana 0, antes de empezar)

Antes de abrir el primer Sprint, el equipo (liderado por M4) debe reunirse y resolver estas decisiones del GDD que bloquean la implementación:

- Stats numéricas base de los 12 personajes (ATK, MAGIA, PV, DESTREZA por clase y rareza)
- Número de runas genéricas y efecto de cada una (bonus de qué stat y cuánto)
- Valores de recompensas por nodo (tickets y moneda premium por rango de nodo)
- Capacidad máxima de pociones y tiempo de regeneración

Miembro	Tareas Sprint 1
M1	Diseñar y crear esquema SQLite completo. Implementar seed de datos de personajes, armas y runas. Funciones get/set básicas.
M2	Implementar fórmulas de daño (Guerrero, Mago, Asesino). Motor de combate (simulación + generación de log). Tests unitarios de las fórmulas.
M3	Pantalla de selección de facción. Pantalla Home con personaje activo. Navegación básica entre pantallas. Primeros sprites de personajes B.
M4	Configurar repositorio y entorno compartido. Crear proyecto Firebase y estructura de datos de enemigos para los nodos 1-5. Resolver decisiones del GDD pendientes.

✓ Entregable para revisión: App arranca, muestra selección de facción, navega al Home con personaje B de la facción elegida. Combate simulable en consola/tests.

SPRINT 2 Días 15–28

□ **Objetivo:** Gacha funcional, inventario y primeros nodos del mapa

Miembro	Tareas Sprint 2
M1	Implementar persistencia de pity (contador por banner). Persistencia de recursos (tickets, moneda, pociones). Progreso del mapa (estados de nodo). API interna estable para M2.
M2	Sistema gacha completo: rates, pity (forzar S en tirada N), separación de banners. Gestión de recompensas al completar nodo. Lógica de consumo de pociones.
M3	Pantalla de gacha (2 banners, botón tirada x1, animación de resultado). Pantalla de inventario (lista de personajes y armas). Primeros 5 nodos del mapa (visual).
M4	Poblar Firebase con enemigos nodos 1-10. Integrar M1↔M2 (gacha + persistencia). Primeros tests funcionales end-to-end del flujo gacha→inventario.

✓Entregable para revisión: Gacha funciona (tirar, recibir personaje, ver en inventario). Pity persiste entre sesiones. Mapa con primeros nodos navegables.

SPRINT 3 Días 29–42

□ **Objetivo: Combate completo con animación y sistema de runas**

Miembro	Tareas Sprint 3
M1	Integración Firebase (lectura de stats de enemigos). Gestión completa del equipamiento (arma + 2 runas por personaje). Validación arma S ↔ personaje S.
M2	Integrar stats de runas y armas en el cálculo de stats finales antes del combate. Conectar motor de combate con datos de Firebase. Lógica de equipo/desequipo.
M3	Pantalla de combate completa: animación 2D estilo Darkest Dungeon usando el log. Pantalla de resultado con recompensas. Arte de runas y primeros enemigos.
M4	Integrar M2↔M3 (log de combate → animación). Tests del combate con distintas clases y builds de runas. Balanceo inicial de dificultad de nodos.

✓Entregable para revisión: Flujo completo jugable: elegir facción → gacha → equipar personaje con arma y runas → entrar a nodo → ver combate animado → recibir recompensas.

SPRINT 4 Días 43–56

□ **Objetivo: Contenido completo: todos los personajes, nodos y pulido de sistemas**

Miembro	Tareas Sprint 4
M1	Completar seed de datos con los 12 personajes, 7 armas y todas las runas definitivos. Optimizar consultas SQLite. Historial de tiradas gacha.
M2	Implementar efectos especiales de las 4 armas S y 4 runas únicas S. Afinar fórmulas con datos reales de balance. Misiones diarias (moneda premium).
M3	Arte completo: sprites de los 12 personajes, todos los enemigos, fondos por pantalla, efectos de impacto. Sistema de estrellas visual por nodo. Historial de tiradas (UI).
M4	Poblar Firebase con todos los nodos (mínimo 10-15). Tests de regresión completos. Verificar flujo de demo para tribunal (pity a 5 → S → combate avanzado).

✓Entregable para revisión: Juego con contenido completo. Todos los personajes obtenibles. Mapa completo. Armas y runas únicas S funcionando.

SPRINT 5 Días 57–70

□ **Objetivo: Pulido final, QA, documentación y preparación de la demo**

Miembro	Tareas Sprint 5
M1	Revisión y limpieza del código de BD. Asegurar que la BD no tiene datos corruptos. Documentar esquema final. Backup/export de la BD para entrega.

Miembro	Tareas Sprint 5
M2	Revisión y limpieza de la lógica de juego. Asegurar que el fichero de configuración de rates y pity es fácilmente ajustable. Documentar todas las fórmulas.
M3	Pulido visual: animaciones fluidas, transiciones entre pantallas, feedback de UI (sonidos si da tiempo). Revisión de arte final y consistencia de estilo.
M4	QA final: sesión completa de juego de principio a fin sin bugs bloqueantes. Preparar script de demo para el tribunal. Completar memoria del TFG (sección técnica).

✓Entregable final: APK instalable + código fuente limpio + demo preparada para el tribunal. Flujo gacha → S en 5 tiradas → combate avanzado → resultado demostrable en ~10 minutos.

4. CALENDARIO VISUAL

Sprint	Fechas	Hito al cierre	Responsable revisión
Sprint 1	Días 1–14	BD funcional + selección de facción + combate en tests	Todos · Revisión profesor
Sprint 2	Días 15–28	Gacha completo + inventario + mapa 5 nodos	Todos · Revisión profesor
Sprint 3	Días 29–42	Combate animado + runas + flujo completo jugable	Todos · Revisión profesor
Sprint 4	Días 43–56	Contenido completo (12 personajes, 10+ nodos, arte)	Todos · Revisión profesor
Sprint 5	Días 57–70	Juego pulido, demo lista, documentación TFG	Todos · Entrega final

5. DEPENDENCIAS CRÍTICAS ENTRE MIEMBROS

Estas dependencias deben resolverse en el orden indicado para no bloquear el trabajo de otros miembros:

Quién necesita	Qué necesita	De quién	Cuándo debe estar listo
M2 (gacha)	Funciones de BD: guardar pity, guardar personaje obtenido	M1	Sprint 2 · semana 1
M2 (combate)	Función de lectura de stats de enemigo por nodo desde Firebase	M1 + M4	Sprint 3 · semana 1
M3 (inventario)	Lista de personajes del jugador con stats y equipo actual	M1	Sprint 2 · semana 2
M3 (combate)	Log de combate {turno, atacante, daño, PV_restantes}	M2	Sprint 3 · semana 1
M3 (gacha UI)	Resultado de tirada: rareza + ítem obtenido	M2	Sprint 2 · semana 1
M4 (balanceo)	Fórmulas de daño implementadas y testeadas	M2	Sprint 1 · fin
M4 (integración)	API interna de datos estable y documentada	M1	Sprint 2 · inicio

6. DECISIONES ABIERTAS — PRIORIDAD DE RESOLUCIÓN

Del apartado 10 del GDD, estas son las decisiones que el equipo debe resolver ANTES del Sprint 2 (idealmente en la semana 0):

Decisión	Bloquea a	Responsable de proponer	Fecha límite
Stats base de los 12 personajes (ATK, MAGIA, PV, DESTREZA)	M1 (seed BD), M2 (fórmulas)	M2 + M4	Semana 0
Porcentajes de rates gacha (B / A / S)	M2 (sistema gacha)	M2 + M4	Semana 0
Número de runas genéricas y efecto de cada una	M1 (seed), M2 (stats finales)	M2 + M4	Semana 0
Efectos especiales de las 4 runas únicas S	M2 (efectos especiales)	M2 + M4	Sprint 4
Efectos especiales de las 4 armas únicas S	M2 (efectos especiales)	M2 + M4	Sprint 4
Valores DEF de enemigos por nodo	M4 (Firebase), M2 (fórmulas)	M4	Sprint 1
Valores de recompensas por nodo (tickets + moneda)	M2 (recompensas), M1 (seed)	M4	Semana 0
Cap de pociones y tiempo de regeneración	M2 (pociones)	M2 + M4	Semana 0

7. NOTAS FINALES

Configuración para la demo del tribunal

El GDD ya contempla que el pity esté fijado a 5 tiradas para la demo. Este valor se debe configurar en Firebase para poder cambiarlo sin recompilar. M4 es el responsable de asegurar que esta configuración funciona y de preparar el script de demostración (≈10 minutos).

Repositorio y control de versiones

Se recomienda usar ramas por miembro (feat/M1-database, feat/M2-combat, feat/M3-ui, feat/M4-integration) y hacer merge a main al cierre de cada Sprint antes de la revisión con el profesor. M4 gestiona los merges y resuelve conflictos.

Arte en el TFG

Todo el arte es propiedad del equipo (creado por M3). La estética sigue la referencia Darkest Dungeon en vista lateral 2D. Los nombres y diseños definitivos de personajes son una decisión secundaria que no bloquea el Sprint 1 pero debe estar cerrada antes del Sprint 4.